

Schema Graph 制約型 Text-to-SQL による材料データベース自然言語インターフェース：Rule-based・LLM 生成のカスケードアーキテクチャ— OQMD 金属間化合物 909 件（B2・L1₂ 型）での 7 条件比較検証 —

皆本 悟^{a,*}

^a 国立研究開発法人 物質・材料研究機構（NIMS）、〒305-0047 茨城県つくば市千現 1-2-1、日本

Abstract

Open Quantum Materials Database (OQMD)、Materials Project、AFLOW などの材料データベースには、第一原理計算による膨大な物性データが蓄積されているが、正規化されたリレーショナルスキーマを通じたデータアクセスには多くの材料研究者が持ち合わせていない SQL (Structured Query Language) の知識が必要である。本研究では、正規化された材料データベースに対し、自然言語クエリを実行可能な SQL 文へ自動変換する **Schema Graph** 制約型 **Text-to-SQL** (**T2SQL**) 手法を提案する。

本手法は 6 つの技術要素から構成される：(1) 日英バイリンガル材料用語辞書を備えたドメイン特化型条件抽出器（数値条件パーサー・化学式パーサーを含む）、(2) 外部キー（FK）関係を NetworkX グラフとして表現し、Steiner 木近似により最小 JOIN 経路を計算する **Schema Graph** 走査エンジン、(3) 成功した NL-SQL ペアを蓄積し、TF-IDF コサイン類似度で検索して LLM プロンプトに注入する **Few-Shot** 検索拡張生成（**RAG**）ストア（SQL-as-Few-Shot-Examples。LaTeX 原稿からの種事例自動抽出を含む）、(4) キーワードブラックリスト、単一文制約、テーブルホワイトリスト、カラムホワイトリスト、JOIN 検証、自動 LIMIT 注入を行う多層 **SQL** ガード、(5) 未認識語彙を検出し Rule-based/LLM/明確化要求を切り替える **Coverage Score**、および (6) VASP/DFT ワークフロー質問を SQL 生成前に排除する **Intent Classifier**。

検証データベースとして、正規化リレーショナルスキーマの好例である

*Corresponding author

Email address: minamoto.satoshi@nims.go.jp (皆本 悟)

OQMD 金属間化合物 909 件 (B2 型 CsCl 636 件、L1₂ 型 AuCu₃ 273 件) を採用した。組成・構造・熱力学的安定性が外部キーで接続された 7 テーブル構成は、材料データベースに典型的なスキーマ複雑性を備えており、本手法が他の正規化 RDB (Materials Project、AFLOW、機関内 DB 等) に展開可能であることの実証基盤となる。57 件のキュレーションクエリ、100 件の VASP フォーラム着想ストレステスト、RAG アブレーション (4 条件 × 50 クエリ)、LLM 再現性テスト (20 クエリ × 5 回) を含む包括的実験を実施した。7 条件比較 (Naive Rule-based / LLM-only / LLM+schema / LLM+schema+few-shot / Schema Graph+Rule-based / Schema Graph+LLM / Schema Graph+LLM+RAG) により、Schema Graph 制約を備えた条件はすべて実行成功率 **100%** を達成した (gpt-5.5 使用)。RAG アブレーションでは 4 条件すべてが 100% を達成し (天井効果)、Schema Graph 制約が SQL 品質の主要因であり、Few-Shot 事例の追加効果は限定的であることを示した。VASP フォーラム着想ストレステストでは、Intent Classifier 導入により out-of-scope 正解率が 0% から 100% に改善した。

Rule-based モードは外部 API 依存ゼロで **32–60 ms** のレイテンシを実現し、gpt-5.5 モードは **2.4–6.1** 秒 (49–194 倍遅延) の代償で辞書未登録語彙・数値フィルタ・否定条件への対応力を提供する。

AI for Science (AI4S) のパラダイムにおいて、構造化された材料データへの自然言語アクセスは自律 AI 駆動型材料探索ワークフローの前提条件である。本手法は材料インフォマティクスにおけるデータ活用の民主化に貢献し、AI4S 駆動型材料探索の基礎的構成要素を提供する。

Keywords: Text-to-SQL, 材料データベース, スキーマグラフ, 検索拡張生成 (RAG), OQMD, 金属間化合物, 自然言語処理, マテリアルズ・インフォマティクス, AI for Science

Contents

1	緒言	5
1.1	AI for Science と材料インフォマティクスにおけるデータアクセス障壁	5
1.2	Text-to-SQL の技術動向	6
1.3	材料インフォマティクスにおける NLP	6
1.4	目的と新規性	6
2	手法	7
2.1	材料データのためのデータベーススキーマ設計	7

2.1.1	OQMD データの 7 テーブルスキーマへの正規化	7
2.1.2	RAG コンテキスト注入のための XML/YAML 表現 . . .	10
2.2	材料用語辞書を備えた条件抽出器	10
2.2.1	辞書構造	10
2.2.2	Unicode 正規化とパターンマッチング	10
2.2.3	出力形式	10
2.2.4	数値条件パーサー	11
2.2.5	化学式パーサーと曖昧性処理	11
2.2.6	Coverage Score : 未認識語彙検出	12
2.2.7	Intent Classifier : スコープ外質問の事前排除	12
2.3	Schema Graph 走査エンジン	13
2.3.1	PostgreSQL メタデータからのグラフ構築	13
2.3.2	最小被覆 JOIN 経路 : Steiner 木近似	13
2.3.3	JOIN 経路 → SQL FROM/JOIN 句生成	15
2.3.4	多元素 AND クエリ : EXISTS 副問い合わせパターン . . .	15
2.3.5	比較 : Naive vs. Schema Graph (具体例)	15
2.4	SQL 生成 : Rule-based モードと LLM モード	15
2.4.1	Rule-based モード (Level 1)	16
2.4.2	Few-Shot RAG を備えた LLM モード (Level 2)	16
2.5	Few-Shot RAG ストア (SQL-as-Few-Shot-Examples)	17
2.5.1	アーキテクチャ	17
2.5.2	材料クエリのための TF-IDF トークナイゼーション . . .	17
2.5.3	研究論文からの種事例抽出	17
2.6	SQL ガード : 多層安全性検証	18
2.6.1	基本安全性検証 (層 1-5)	18
2.6.2	カラムホワイトリスト検証 (層 6)	19
2.6.3	FK 整合 JOIN 検証 (層 7)	19
2.6.4	ガード判定分類 (層 8)	19
2.7	推奨運用構成 : カスケードモード	19
3	データ準備	20
3.1	検証データベースとしての OQMD	20
3.2	RDB へのデータ投入	20
3.3	スキーマ拡張	20
4	結果	21
4.1	テストケース設計	21
4.1.1	キュレーション回帰テスト (80 件)	21

4.1.2	Baseline 比較実験 (7 条件 × 57 クエリ)	21
4.1.3	VASP フォーラム ストレス テスト (100 クエリ)	21
4.1.4	RAG アブレーション (4 条件 × 50 クエリ)	22
4.1.5	LLM 再現性 テスト (20 クエリ × 5 回)	22
4.2	7 条件 Baseline 比較	22
4.3	Rule-based vs. LLM 詳細比較	23
4.3.1	注目すべき乖離	23
4.3.2	スケーラビリティとレイテンシ	23
4.4	データパイプライン結合テスト	24
4.5	VASP フォーラム ストレス テスト 結果	25
4.6	RAG アブレーション 結果	26
4.7	LLM 再現性 テスト 結果	26
4.8	SQL インジェクションと安全性 結果	27
5	考察	27
5.1	多次元評価	27
5.2	各モードの利点と欠点	27
5.3	限界と制約	27
5.3.1	テストケースの自己参照性 (深刻度: 高)	27
5.3.2	RAG 天井効果 (深刻度: 中 → 解決済み)	28
5.3.3	無通知失敗 (深刻度: 中 → 緩和済み)	28
5.3.4	Out-of-scope 検出 (深刻度: 中 → 部分解決)	28
5.3.5	小規模スキーマ (深刻度: 中)	28
5.3.6	LLM 再現性 (深刻度: 低)	29
5.4	関連システムとの比較	29
5.5	AI for Science (AI4S) における意義	30
5.6	今後の展望	30
6	結論	31
A	全テストケース一覧	36
B	生成 SQL 例	37
B.1	A01: 「Fe を含む B2 化合物を出して」	37
B.2	A03: 「Ni と Al の両方を含む化合物を出して」	38

1. 緒言

1.1. *AI for Science* と材料インフォマティクスにおけるデータアクセス障壁

AI for Science (AI4S) —科学的発見を加速するための人工知能の体系的応用—は、材料研究を変革しつつある [1]。AI4S は、AI エージェントが自律的に仮説を生成し、大規模データベースから関連データを検索し、実験やシミュレーションを設計し、結果を解釈する統合的ワークフローを構想している。このビジョンにおいて、構造化された材料データへの自然言語アクセスは単なる利便性ではなく前提条件である：AI エージェント（またはそれを指揮する研究者）が正規化リレーショナルデータベースを効率的にクエリできなければ、AI4S パイプライン全体がデータ検索段階でボトルネックとなる。

ハイスループット第一原理計算の急速な発展により、複数の大規模材料データベースが構築されている：Open Quantum Materials Database (OQMD、約 100 万件) [2, 3]、Materials Project (15 万件超) [4]、AFLOW (350 万件超) [5]、NOMAD [6]。これらのデータベースは計算材料探索、候補相のスクリーニング [7, 8]、実験観測の検証に不可欠である。

しかし、正規化リレーショナルスキーマ（外部キーで接続された複数テーブル）に格納されたデータへのアクセスには SQL 知識が必要である。例えば、「Fe を含む安定な B2 化合物の格子定数」を適切に正規化されたスキーマで検索するには、元素・プロトタイプ・熱力学的安定性に対する WHERE 条件を持つ 4 テーブル JOIN が必要となる—データベース技術者にとっては容易だが、材料研究者にとっては非自明な操作である。

多くの材料データベースが提供する REST API や GraphQL エンドポイントはこの障壁を部分的に緩和するが、RDB 全般に共通する根本的制約が残る：(a) 事前定義されたフィルタ構文への依存（アドホックな複合条件や集約が困難）、(b) スキーマ更新時の API 非互換リスク、(c) 機関内データベースや独自拡張スキーマには API が存在しない場合が多いこと。T2SQL アプローチは、スキーマ構造を実行時に解析するため、API の有無に関わらず任意の正規化 RDB に対して原理的に展開可能である。

AI4S の文脈において、このデータアクセス障壁は特に深刻である：材料スクリーニングや物性予測を行う自律 AI エージェントは、複雑な複合条件データベースクエリをプログラマ的に発行する必要がある。正しい SQL を確実に生成する自然言語インターフェースは、人間の研究者と AI エージェントの双方がデータベース工学の専門知識なしに材料データベースと対話することを可能にし、AI4S ワークフローの重要なボトルネックを除去する。

1.2. Text-to-SQL の技術動向

Text-to-SQL (T2SQL) 研究は大規模ベンチマークにより急速に進展している：Spider [9] (10,181 問、200 データベース、138 ドメイン)、BIRD [10] (12,751 問、95 データベース、33.4 GB)。最近の LLM ベース手法は高い精度を達成している：DAIL-SQL [11] はスケルトンベースの few-shot 事例選択により Spider 上で実行精度 86.6% を報告し、DIN-SQL [12] は分解型文脈内学習により 85.3% を達成している。(author?) [13] はスキーマリンキングの支配的役割を指摘する包括的サーベイを提供し、(author?) [14] は高度な LLM により明示的スキーマリンキングが不要になる可能性を論じているが、これは議論が続いている。

しかし、既存の **T2SQL** ベンチマークはすべて汎用ドメイン (e コマース、医療、教育) を対象としており、材料科学データベース固有の課題に取り組んだ先行研究は存在しない：ドメイン固有の用語 (Strukturbericht 記号、プロトタイプ名)、バイリンガルクエリ (日英元素名)、組成テーブル正規化 (化合物ごとに元素 1 行)、熱力学的安定性フィルタ ($E_{\text{hull}} \leq 0.001 \text{ eV/atom}$) などの課題である。

1.3. 材料インフォマティクスにおける NLP

材料科学における NLP 応用は拡大している：MatSci-NLP [15] は材料テキストの 7 つの NLP タスクをベンチマークし、LLAMAT [16] は LLaMA の継続事前学習により情報抽出を改善し、Materials Knowledge Navigation Agent (MKNA) [17] は自然言語の意図をデータベース検索・構造生成アクションに変換し、OptiMat Alloys [18] は LLM 駆動型対話式合金探索ツールを提供している。Materials Project は MCP ベースの LLM 統合の提供を開始している。

これらの研究は情報抽出やエージェントベースのデータベース対話を扱っているが、正規化リレーショナルデータベース上のスキーマ認識型 **SQL** 生成—特に外部キーグラフ走査による自動 JOIN 経路探索—を対象としたものはない。このギャップが本手法の焦点である。

1.4. 目的と新規性

本研究の貢献は以下の 5 点である：

1. **Schema Graph 制約型 T2SQL** 手法：日英バイリンガル材料用語辞書と FK 関係グラフ走査を組み合わせ、正規化材料データベース上の自動 JOIN 経路計算を実現する。7 条件比較実験により、Schema Graph 制約が実行成功率 100% の主要因であることを示す。

2. **6層安全パイプライン**: Coverage Score (未認識語彙検出)、Intent Classifier (スコープ外質問検出)、数値条件パーサー、化学式パーサー、多層 SQL ガード (カラム WL・JOIN 検証) を統合し、Silent Constraint Dropping (未登録語彙の無通知破棄) を防止する。
3. **RAG** アブレーション研究: 4 条件 (no examples / manual / paper / all) × 50 クエリの比較で、gpt-5.5 では Schema Graph 制約のみで天井効果が生じ、Few-Shot 事例の追加効果が消失する条件を定量的に示す。
4. **100 クエリ VASP** フォーラムストレステスト: SQL-answerable、数値条件、曖昧、スコープ外、安全性の 5 カテゴリにわたる現実的な材料研究者クエリを模擬した大規模評価。
5. カスケード **Rule-based** → **LLM** アーキテクチャの定量評価: Rule-based 32–60 ms vs gpt-5.5 2.4–6.1 秒 (49–194 倍) のレイテンシ・正確性トレードオフを実測。

2. 手法

本節では各技術要素を詳述する。システム全体のパイプラインアーキテクチャを図 1 に示す。

Fig. 1: System Architecture — Schema-Graph-Assisted Text-to-SQL Pipeline

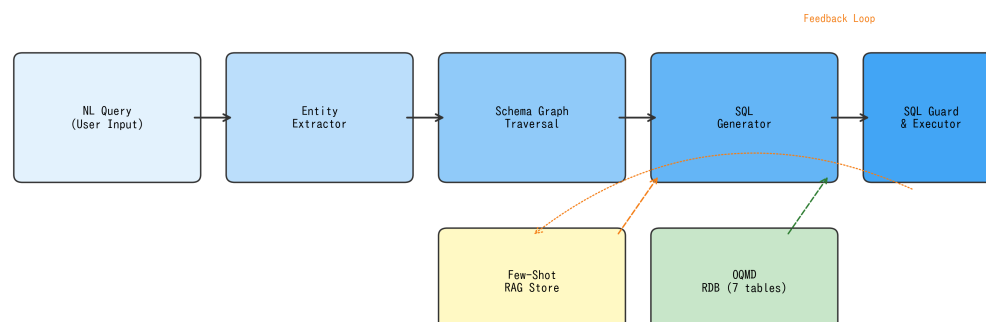


Figure 1: Schema Graph 支援 Text-to-SQL パイプラインのシステムアーキテクチャ。実線矢印は主要データフロー、破線矢印は Few-Shot RAG フィードバックループ（成功したクエリペアが蓄積・検索される）を示す。SQL ガードはデータベース実行前の最終安全層として機能する。

2.1. 材料データのためのデータベーススキーマ設計

2.1.1. OQMDデータの7テーブルスキーマへの正規化

OQMDは組成・構造・熱力学的安定性を体系的に格納しており、材料データベースに典型的なスキーマ複雑性（多対多関係、正規化された組成テーブル、構造パラメータの分離）を備えた好例である。APIから取得したフラットなJSONレコードを7テーブルのリレーショナルスキーマに正規化する（図2、表1）。設計はBoyce-Codd正規形（BCNF）に従い、以下の材料ドメイン固有の考慮に基づく。

Fig. 2: Entity-Relationship Diagram — Materials Database Schema (7 Tables)

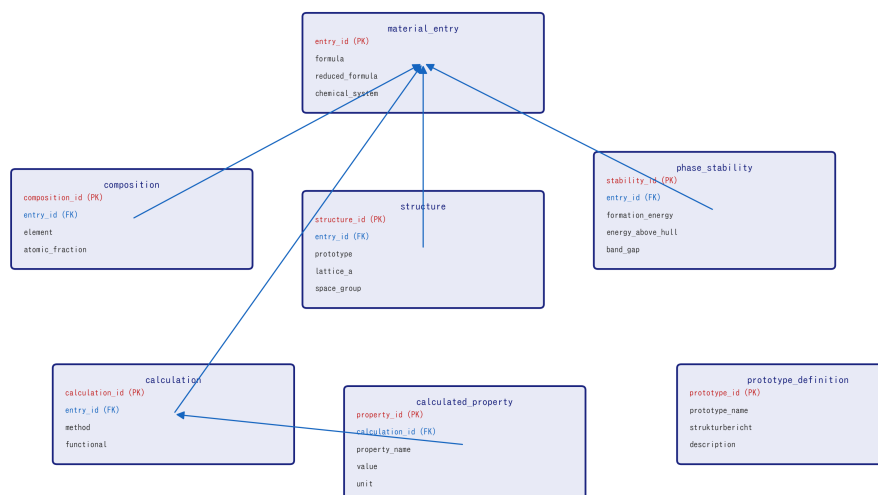


Figure 2: 7テーブル材料データベーススキーマの Entity-Relationship 図。すべてのテーブルは外部キー（青線）により material_entry に接続される。赤=主キー、青=外部キー。

組成テーブルの分離。．単一の化合物が複数の元素を含む場合がある（例： $\text{Fe}_3\text{Al} \rightarrow \text{Fe}$ と Al ）。各元素を composition テーブルの個別行として格納することで、文字列解析なしに柔軟な元素ベースクエリ（AND, OR, NOT）を可能にする。ただし、この正規化は重要な複雑性を導入する：「Ni と Al の両方を含む化合物」のクエリは単純な `WHERE element = 'Ni' AND element = 'Al'` では不可能である（単一行が両条件を同時に満たすことはないため0行が返る）。代わりに EXISTS 副問い合わせが必要となる（2.3.4 節参照）。

構造/相安定性/計算の分離。．結晶構造情報（プロトタイプ、格子定数）、熱力学的安定性（hull 上エネルギー）、計算メタデータ（DFT 汎関数）を別テーブルに格納し、独立した更新・拡張を可能にする。

Table 1: データベーススキーマ概要 (7 テーブル)。

テーブル名	主キー	外部キー	主要カラム
material_entry	entry_id	—	formula, reduced_formula, chemical_system
composition	composition_id	entry_id	element, atomic_fraction
structure	structure_id	entry_id	prototype, strukturbericht, lattice_a, space_group
phase_stability	stability_id	entry_id	formation_energy_per_atom, energy_above_hull, band_gap
calculation	calculation_id	entry_id	method, functional
calculated_property	property_id	calculation_id	property_name, value, unit
prototype_definition	prototype_id	—	prototype_name, strukturbericht, description

計算物性の *Entity-Attribute-Value* (EAV) パターン。 `calculated_property` テーブルは EAV パターン (`property_name`, `value`, `unit`) を使用し、スキーマ変更なしに任意の計算物性 (体積弾性率、せん断弾性率等) を収容する。

2.1.2. RAG コンテキスト注入のための XML/YAML 表現

スキーマ構造は YAML (`allowed_schema.yaml`) としてシリアル化され、二重の目的に供される: (a) SQL ガードのテーブル/カラムホワイトリスト (2.6 節)、(b) LLM プロンプトへの RAG コンテキスト—スキーマ YAML がシステムメッセージに注入され、LLM が利用可能なテーブル・カラム・型を理解する。このアプローチは汎用 T2SQL のスキーマシリアライゼーション技法 [11] と類似するが、材料データベースのドメイン固有カラム意味論 (例: `energy_above_hull` は E_{hull} を表す) に特化している。

2.2. 材料用語辞書を備えた条件抽出器

条件抽出器は、YAML ベースの日英バイリンガル用語辞書 (`material_terms.yaml`) を用いて、自然言語クエリを構造化された条件辞書に変換する。

2.2.1. 辞書構造

辞書は 4 つのカテゴリを含む:

1. プロトタイプ別名: プロトタイプ名とその変種の対応。

例: B2 $\leftrightarrow$ [B2, CsCl, CsCl 型, B2 型]

L12 $\leftrightarrow$ [L1₂, L12, AuCu₃, Cu₃Au 型, γ']

2. 元素マッピング: 元素記号 $\leftrightarrow$ 日英名対応。

例: Fe $\leftrightarrow$ [Fe, 鉄, iron]

3. 安定性用語: 自然言語 $\rightarrow$ 数値閾値。

「安定」/ “stable” $\rightarrow$ `energy_above_hull` $\leq$ 0.001 eV/atom

「準安定」/ “metastable” $\rightarrow$ `energy_above_hull` $\leq$ 0.05 eV/atom

4. 物性用語: 物性記述からテーブル. カラム参照への対応。

「格子定数」/ “lattice constant” $\rightarrow$ `structure.lattice_a`

「形成エネルギー」/ “formation energy” $\rightarrow$ `phase_stability.formation_energy_per_atom`

2.2.2. Unicode 正規化とパターンマッチング

Unicode 下付き文字 (ⒶⒷⒸⒹⒺⒻⒼⒽⒾⒿ) はマッチング前に ASCII 数字に正規化される。元素記号認識はワード境界正規表現を使用し、独立した「Fe」を「FeCoNi」部分文字列から正しく区別する。

2.2.3. 出力形式

抽出器は構造化辞書を返す：

Listing 1: 条件抽出の例。

```
1 # 入力: "Feを含む安定なB2化合物を出して"  
2 # 出力:  
3 {  
4   "prototype": "B2",  
5   "contains_elements": ["Fe"],  
6   "stability": "stable",  
7   "sort_by": None,  
8   "sort_order": None  
9 }
```

2.2.4. 数値条件パーサー

条件抽出器は、自然言語中の数値比較表現を構造化された WHERE 句条件に変換する正規表現ベースの数値条件パーサーを含む。対応する5種類のパターンを表 2 に示す。

Table 2: 数値条件パーサーの対応パターン。

パターン種別	入力例	出力
記号比較	band_gap > 1.0 eV	WHERE band_gap > 1.0
日本語後置	形成エネルギーが0.5 以上	WHERE ... >= 0.5
日本語より構文	格子定数が3.0 より大きい	WHERE lattice_a > 3.0
符号判定	形成エネルギーが負	WHERE ... < 0
範囲指定	band gap 0.5–2.0 eV	WHERE ... BETWEEN 0.5 AND 2.0

物性名から対応カラムへのマッピングは内部辞書(`_PROPERTY_COLUMN_MAP`)により管理され、「格子定数」→`structure.lattice_a`、「形成エネルギー」→`phase_stability.formation_energy_per_atom`等の変換を行う。単位変換(eV, Å, GPa等)も内蔵するが、現スキーマではOQMDの内部単位がそのまま格納されているため、変換係数は全て1.0である。

2.2.5. 化学式パーサーと曖昧性処理

化学式パーサーは、クエリ中の化学式トークン(例:`Ni3Al`、`NiAl`、`Fe2CoAl`)を検出し、以下の2種類の解釈に分類する：

- **exact_formula**：化学量論比が明示された場合(例：`Ni3Al` → `formula = 'Ni3Al'`)。組成テーブルの化学式カラムとの完全一致検索を生成する。

- **contains_elements** : 化学量論比が省略された場合 (例 : NiAl $\rightarrow$ 元素 Ni AND 元素 Al)。組成テーブルに対する EXISTS 副問い合わせ (多元素 AND) を生成する。

判定基準は、解析された **composition** 辞書において全元素の係数が 1.0 でないものが存在するか否かである。この区別は重要であり、 NiAl を **exact_formula** と誤判定すると Ni_3Al 等の化合物が結果から欠落する。

2.2.6. Coverage Score : 未認識語彙検出

Coverage Score は、自然言語クエリに含まれる意味的トークンのうち、用語辞書・元素辞書・数値パターンで認識された割合を $[0, 1]$ で定量化する。この指標は、Silent Constraint Dropping (未登録語彙の無通知破棄) を防止するために導入された。

算出手順 :

1. クエリを正規表現でトークン化し、ストップワード (助詞、接尾辞等) を除去
2. 各トークンを辞書エイリアス (プロトタイプ、元素、物性、安定性) と照合
3. 元素記号の既知/未知を判定 (辞書登録済みか否か)
4. $\text{Coverage} = \text{認識トークン数} / \text{全意味トークン数}$

Coverage Score に基づき、システムは 3 段階のアクションに分岐する :

- ≥ 0.8 (かつ未知元素なし) $\rightarrow$ **execute_rule_based** : Rule-based モードで安全に実行
- ≥ 0.5 (または未知元素あり) $\rightarrow$ **fallback_to_llm** : LLM モードにエスカレーション
- $< 0.5 \rightarrow$ **clarification_required** : ユーザーに明確化を要求

この仕組みにより、「Xe を含む化合物」のようなクエリで Xe (キセノン) が辞書未登録の場合、Coverage Score が低下し、条件を無視して SQL 生成するのではなく、LLM フォールバックまたは明確化要求が発生する。

2.2.7. Intent Classifier : スコープ外質問の事前排除

VASP フォーラムストレステスト (4.5 節) において、gpt-5.5 が VASP ワークフロー質問に対して SQL を生成してしまう問題が判明した (out-of-scope 正解率 0%)。この問題に対処するため、SQL 生成パイプラインの前段にルールベースの Intent Classifier を導入した。

Intent Classifier は以下の 5 カテゴリにクエリを分類する :

1. **db_query** : 材料データベースで回答可能
2. **out_of_scope** : VASP/DFT ワークフロー、計算手法、一般知識
3. **ambiguous** : 明確化が必要
4. **unsafe** : SQL インジェクション等の不正意図

5. greeting : 挨拶・雑談

分類アルゴリズムは、20 個の VASP/DFT ワークフローパターン (INCAR, KPOINTS, POTCAR 等のファイル名、SCF 収束、ENCUT 等のパラメータ名、フォノン計算、Wannier90、Bader 解析等) と 9 個の DB 特化パターン (化合物検索、安定性フィルタ、物性条件等) の正規表現マッチング数を比較する。VASP パターンのみが検出された場合は `out_of_scope` として拒否し、DB パターンが優勢な場合は `db_query` としてパイプラインに通過させる。

2.3. Schema Graph 走査エンジン

本手法の中核的アルゴリズム貢献である。このエンジンはデータベーススキーマをグラフとして表現し、最適な JOIN 経路を自動計算する。

2.3.1. PostgreSQL メタデータからのグラフ構築

外部キー関係は PostgreSQL の `information_schema` から動的に抽出される：

Listing 2: FK 関係抽出クエリ。

```
1 SELECT tc.table_name AS source_table,
2       kcu.column_name AS source_column,
3       ccu.table_name AS target_table,
4       ccu.column_name AS target_column
5 FROM information_schema.table_constraints tc
6 JOIN information_schema.key_column_usage kcu
7   ON tc.constraint_name = kcu.constraint_name
8 JOIN information_schema.constraint_column_usage ccu
9   ON ccu.constraint_name = tc.constraint_name
10 WHERE tc.constraint_type = 'FOREIGN KEY'
11 AND tc.table_schema = 'public';
```

結果は NetworkX [19] の無向グラフ $G = (V, E)$ として格納される。ここで $V = \{\text{テーブル名}\}$ 、 $E = \{\text{FK 関係}\}$ である。FK メタデータは実行時に読み取られるため、コード変更なしにスキーマ変更に自動適応する。

2.3.2. 最小被覆 JOIN 経路 : Steiner 木近似

条件抽出器の出力から決定された必要テーブル集合 $R \subseteq V$ に対し、 R のすべてのテーブルを接続する G の最小部分グラフが必要となる。これはグラフにおける Steiner 木問題 [20] であり、一般に NP 困難である。

本スキーマの 7 ノード 6 辺グラフに対し、木構造グラフでは厳密解を与える貪欲ヒューリスティック (アルゴリズム 1) を採用する：

Algorithm 1 最小被覆 JOIN 部分グラフ (Steiner 木近似)。

Require: FK 関係グラフ $G = (V, E)$ 、必要テーブル集合 R

Ensure: JOIN 経路リスト P

```
1:  $P \leftarrow \emptyset$ ,  $\text{covered} \leftarrow \emptyset$ 
2: for all  $(s, t) \in \binom{R}{2}$  do
3:   if  $s \in \text{covered} \wedge t \in \text{covered}$  then
4:     continue ▷ 両端点は既に接続済み
5:   end if
6:    $p \leftarrow \text{ShortestPath}(G, s, t)$  ▷ FK グラフ上の BFS
7:   if  $p \neq \emptyset$  then
8:      $P \leftarrow P \cup \{p\}$ 
9:      $\text{covered} \leftarrow \text{covered} \cup \text{nodes}(p)$ 
10:  end if
11: end for
12: for all  $r \in R \setminus \text{covered}$  do ▷ 孤立必要テーブルの処理
13:    $c \leftarrow \text{covered の最近傍ノード}$ 
14:    $P \leftarrow P \cup \{\text{ShortestPath}(G, r, c)\}$ 
15: end for
16: return  $P$ 
```

計算量。・ $|V| = n$ ノード、 $|R| = k$ 必要テーブルのグラフに対し、アルゴリズムは $O(k^2)$ 回の最短路計算を行い、各計算は $O(n + |E|)$ (BFS) である。本スキーマ ($n = 7$, $k \leq 5$) ではマイクロ秒で完了する。

木構造 FK グラフでの正確性。・ G が木 (本スキーマでは `material_entry` がハブ) の場合、Steiner 木は一意であり、貪欲アルゴリズムはこれを厳密に求める。FK サイクルを持つスキーマ (自己参照テーブル等) では近似が不要な辺を含む可能性があるが、SQL 生成においては DISTINCT を適用すれば正確性に影響しない。

2.3.3. JOIN 経路 $\rightarrow$ SQL FROM/JOIN 句生成

P の各経路は SQL JOIN 句に変換される。FK メタデータが結合カラム名を提供する：

Listing 3: 経路からの JOIN 句生成。

```
# 経路: [material_entry, structure]
# FK: structure.entry_id -> material_entry.entry_id
# 生成:
#   FROM material_entry m
#   JOIN structure s ON s.entry_id = m.entry_id
```

テーブル別名はテーブル名の先頭文字から自動割当される (衝突時は解決処理あり)。

2.3.4. 多元素 AND クエリ: EXISTS 副問い合わせパターン

組成テーブル正規化に起因する材料固有の重要課題が存在する。「Ni と Al の両方を含む化合物」のクエリには以下が必要：

Listing 4: EXISTS 副問い合わせによる多元素 AND クエリ。

```
1 SELECT DISTINCT m.entry_id, m.formula
2 FROM material_entry m
3 WHERE EXISTS (
4     SELECT 1 FROM composition
5     WHERE entry_id = m.entry_id AND element = 'Ni')
6 AND EXISTS (
7     SELECT 1 FROM composition
8     WHERE entry_id = m.entry_id AND element = 'Al')
9 LIMIT 100;
```

Schema Graph エンジンは複数元素が指定された場合を検出し、直接 JOIN の代わりに EXISTS 副問い合わせを自動生成する。このパターンなし (Naive アプローチ) では、WHERE `c.element = 'Ni' AND c.element = 'Al'` は単

一組成行が両条件を同時に満たすことが不可能なため常に 0 行を返す。これが Schema Graph 手法により防止される最も重要な故障モードである。

2.3.5. 比較 : *Naive vs. Schema Graph* (具体例)

表 3 に「Fe を含む B2 化合物を出して」に対する差異を示す：

Table 3: SQL 生成比較 : 「Fe を含む B2 化合物」に対する Naive vs. Schema Graph。

観点	Naive (Level 0)	Schema Graph (Level 1)
JOIN テーブル	常に全 6 テーブル	composition + structure のみ (2 テーブル)
JOIN 数	5 JOIN	2 JOIN
DISTINCT	なし → 重複行	あり
LIMIT	なし → 無制限	あり (100)
安全性検証	なし	完全 (キーワード + テーブル WL)
多元素 AND	不正 (0 行)	正確 (EXISTS)

2.4. SQL 生成 : *Rule-based* モードと *LLM* モード

システムは 2 つの SQL 生成モードをサポートし、いずれも Schema Graph 制約内で動作する。

2.4.1. *Rule-based* モード (*Level 1*)

決定論的テンプレートベース生成器が、抽出された条件と Schema Graph JOIN 経路から SQL を構築する。外部 API は不要で、レイテンシは 32–60 ms (4.3.2 節)。

改訂版での拡張。前稿の Rule-based モードでは数値比較 WHERE 句の生成が不可であったが、改訂版では数値条件パーサー (2.2.4 節) および化学式パーサー (2.2.5 節) を統合し、`band_gap > 1.0`、`formation_energy < 0`、`Ni3Al` (完全一致) 等の条件を Rule-based モード内で処理可能とした。

残存する制限。以下のクエリ種別は Rule-based モードでは処理できず、Coverage Score (2.2.6 節) または Intent Classifier (2.2.7 節) により自動的に LLM フォールバック、明確化要求、またはスコープ外拒否に分岐する：

- 否定条件 (「Fe を含まない化合物」)
- 辞書未登録語彙を含む自由記述クエリ
- VASP/DFT ワークフロー質問 (Intent Classifier が検出・拒否)
- 複雑な集約 (GROUP BY + HAVING 等)

2.4.2. Few-Shot RAG を備えた LLM モード (Level 2)

Rule-based モードの辞書カバレッジが不十分な場合 (未登録元素、数値条件、複雑な表現など)、LLM ベース生成にフォールバックする。LLM (本実験では gpt-5.5、API 設定の詳細は Supplementary S5 節を参照) は以下の構造化プロンプトを受け取る:

1. システムメッセージ: タスク説明 + スキーマ YAML (テーブル名、カラム名、型、FK 関係)
2. **Few-Shot** 事例: RAG ストアからの Top- k 類似 NL-SQL ペア (2.5 節)
3. ユーザーメッセージ: 自然言語クエリ
4. 制約: 「SELECT 文のみ生成」「LIMIT 100 を含める」「スキーマに記載されたテーブルのみ使用」

生成された SQL は実行前に SQL ガード (2.6 節) で検証される。

2.5. Few-Shot RAG ストア (SQL-as-Few-Shot-Examples)

2.5.1. アーキテクチャ

Few-Shot ストアは T2SQL に特化した RAG [21] パターンを実装する:

1. 蓄積: クエリが正常に結果を返した場合、(NL クエリ、SQL、条件、行数、ソース) タプルを JSON ストアに追加する。
2. 検索: 新規クエリに対し、全蓄積 NL クエリとの TF-IDF [22] コサイン類似度を計算し、上位 k 件 (デフォルト $k = 3$) を返す。
3. 注入: 検索された事例を「Question: ... SQL: ...」ブロックとしてフォーマットし、LLM プロンプトの先頭に配置する。

2.5.2. 材料クエリのための TF-IDF トークナイゼーション

標準 NLP トークナイザは化学式や元素記号を不正確に分割するため、材料クエリには不適切である。本トークナイザは複合正規表現パターンを使用する:

Listing 5: 材料クエリ用 TF-IDF トークナイザ。

```
# トークナイゼーションパターン:
# 1. 化学記号: [A-Z][a-z]? (Fe, Ni, Al, ...)
# 2. 日本語文字: [\u3040-\u9fff]+
# 3. 英単語: [a-z]+
# 4. 数字: \d+
tokens = re.findall(
    r'[A-Z][a-z]?|[\u3040-\u9fff]+|[a-z]+|\d+',
    query.lower()
)
```

IDF は $\text{IDF}(t) = \log(N/n_t)$ で計算される。ここで N は蓄積事例総数、 n_t は語 t を含む事例数である。クエリと蓄積事例の TF-IDF ベクトル間のコサイン類似度によりランキングされる。

2.5.3. 研究論文からの種事例抽出

手動キュレーションなしに Few-Shot ストアをブートストラップするため、LaTeX 原稿からの自動種事例抽出を実装した。抽出器は以下のパターンを走査する：

- B2 型パターン： $\text{B2}[-\backslash\text{s}]\text{type} + \text{化学式} \rightarrow \{\text{prototype: B2, elements: 抽出}\}$
- L1₂ 型パターン： $\text{L1}_2 / \text{L12} + \text{化学式} \rightarrow \{\text{prototype: L1}_2, \text{elements: 抽出}\}$
- 100 文字以内のコンテキストキーワード：“lattice”, “stable”, “formation energy”

抽出された各化合物言及に対し、正規 NL クエリと対応 SQL が生成される。本実験では `hea_lattice_prediction.tex` から 4 件の種事例が自動抽出され、手動キュレーション 7 件と合わせて合計 11 件となった（表 4）。

Table 4: Few-Shot ストア構成（11 事例）。

ソース	件数	比率	クエリ例
手動キュレーション	7	64%	「Fe を含む B2 化合物を出して」
論文抽出	4	36%	「B2 FeAl の物性を出して」

2.6. SQL ガード：多層安全性検証

Rule-based モード・LLM モードを問わず、生成された全 SQL はデータベース実行前に 8 層検証パイプラインを通過する。前稿の 5 層構成に対し、カラムホワイトリスト検証（層 6）、FK 整合 JOIN 検証（層 7）、ガード判定分類（層 8）を追加した。

2.6.1. 基本安全性検証（層 1–5）

1. 複数文検出：SQL 本体内にセミコロンがあれば拒否（`SELECT ...`；`DROP TABLE ...` 等のピギーバック攻撃を防止）。
2. 禁止キーワード検出：`INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER`, `TRUNCATE`, `CREATE`, `GRANT`, `REVOKE`, `COPY` をワード境界正規表現で走査。
3. **SELECT** 専用制約：`SELECT` または `WITH` で始まらない文を拒否。

4. テーブルホワイトリスト検証：FROM/JOIN 句の全テーブル名を抽出し、`allowed_schema.yaml`に存在するか検証。未宣言テーブル(例:`secret_passwords`)参照クエリは拒否。
5. 自動 **LIMIT** 注入：LIMIT 句がなければ LIMIT 100 を付加し、無制限結果セットによる過剰メモリ・計算消費を防止。

2.6.2. カラムホワイトリスト検証（層 6）

SELECT 句、WHERE 句、ORDER BY 句で参照される全カラム名を抽出し、`allowed_schema.yaml` のカラム定義と照合する。スキーマに存在しないカラム（例：LLM が幻覚生成した `melting_point`）を参照する SQL は実行前に拒否される。これにより、LLM モードにおけるスキーマ幻覚（hallucinated schema）を防止する。

2.6.3. FK 整合 JOIN 検証（層 7）

FROM/JOIN 句のテーブル結合条件が、PostgreSQL メタデータから取得した FK 関係と整合するかを検証する。Schema Graph 走査（2.3 節）が生成した JOIN 経路と実際の SQL 内の JOIN 条件を比較し、以下を検出する：

- FK 関係に基づかない不正 JOIN 条件（例：`ON a.id = b.name`）
- Schema Graph 経路に含まれないテーブル間の JOIN（不要 JOIN）
- JOIN 条件の欠落（Cartesian product リスク）

2.6.4. ガード判定分類（層 8）

8 層検証の結果に基づき、生成 SQL を以下の 4 カテゴリに分類する：

1. **SAFE**：全層パス。実行可能。
2. **MODIFIED**：自動修正適用（LIMIT 注入等）。修正後実行可能。
3. **BLOCKED**：安全性違反により実行拒否。ユーザーに理由を通知。
4. **WARNING**：軽微な逸脱（カラム幻覚疑い等）。実行するが警告付き。

オプションで SQLGlot [23] が AST（抽象構文木）レベルの構文検証を行う。

2.7. 推奨運用構成：カスケードモード

各生成モードの特性に基づき、以下のカスケード戦略を推奨する：

1. **Intent 分類**: Intent Classifier (2.2.7 節) がクエリを `db_query` / `out_of_scope` / `unsafe` / `greeting` に分類。`db_query` 以外は即座に拒否または適切な応答を返す。
2. 条件抽出 + **Coverage Score**：条件抽出器（2.2 節）がクエリを解析し、Coverage Score（2.2.6 節）で辞書カバレッジを判定。

3. **Rule-based** 生成 (**Coverage** ≥ 0.8): テンプレートベース SQL 生成 (Level 1、32–60 ms)。
4. **LLM** フォールバック (**Coverage** < 0.8): Schema Graph 制約付き LLM 生成にエスカレーション (Level 2、2.4–6.1 秒)。
5. 明確化要求 (**Coverage** < 0.5): ユーザーに追加情報を要求。
6. **SQL** ガード: 生成モードによらず 8 層検証 (2.6 節) を常に適用。
このアプローチは、辞書がカバーするクエリを外部 API 依存なしに処理し、複雑または新規なクエリにのみ LLM 呼び出しを使用する。

3. データ準備

3.1. 検証データベースとしての OQMD

本研究では、材料データベースの正規化 RDB として良く整備された好例である OQMD を検証対象に選定した。OQMD は組成・構造・熱力学的安定性を体系的に提供しており、外部キーで接続された複数テーブル (組成、構造、安定性、計算条件) への正規化が自然に行える。この構造は Materials Project、AFLOW 等の他の材料データベースにも共通しており、OQMD での検証結果は他の正規化 RDB への展開可能性を示唆する。

データは OQMD RESTful API (<https://oqmd.org/oqmdapi/formationenergy>) から、2 種類の金属間化合物プロトタイプ構造について取得した:

- **B2** (**CsCl** 型): フィルタ `prototype=CsCl`。重複除去後 636 の固有組成。安定 ($E_{\text{hull}} \leq 0.001$ eV/atom): 185 件。準安定 ($E_{\text{hull}} \leq 0.05$ eV/atom): 198 件。
- **L1₂** (**AuCu₃** 型): フィルタ `prototype=AuCu3`。273 の固有組成。安定: 88 件。準安定: 138 件。

合計: 909 の固有化合物。各エントリについて 6 フィールドを取得: `name` (化学式)、`entry_id`、`prototype`、`delta_e` (原子当たり形成エネルギー)、`stability` (hull 上エネルギー)、`band_gap`。

API はページネーションされた JSON レスポンス (`limit=200`, `offset=0,200,...`) を返す。重複エントリ (同一化学式、異なる `entry_id`) は最低エネルギーのものを保持して重複除去した。

3.2. RDB へのデータ投入

各 OQMD エントリを 7 テーブルスキーマに分解する。例えば、Fe₃Al (L1₂ プロトタイプ) は以下を生成する: `material_entry` に 1 行、`composition` に 2 行 (Fe: 0.75, Al: 0.25)、`structure` に 1 行 (`prototype=L12`, `lattice_a`, `space_group`)、`phase_stability` に 1 行 (`formation_energy`, `energy_above_hull`, `band_gap`)、`calculation` に 1 行 (`method=GGA`)。

3.3. スキーマ拡張

元スキーマに含まれない OQMD フィールドを収容するため以下のカラムを追加した: `phase_stability` に `band_gap` (REAL), `structure` に `space_group` (TEXT)。材料用語辞書に対応用語 (`band_gap`, `volume_per_atom`, `space_group`) を追加した。

4. 結果

4.1. テストケース設計

正確性、安全性、頑健性を評価するため、多層的なテスト体系を設計した。

4.1.1. キュレーション回帰テスト (80 件)

開発安全性テストとして 6 カテゴリ 39 件の回帰テスト、SQL 検証テスト 6 件、Coverage Score・パーサーテスト 15 件、Intent Classifier テスト 20 件の計 80 件を構築した (表 5)。これらは開発者が設計した回帰テストであり、精度評価のための独立テストではない。

Table 5: 回帰テストカテゴリ (全 80 件)。

カテゴリ	ID	n	検証焦点
正常系	A01–A15	15	元素、プロトタイプ、安定性、ソート、複合条件
該当なし	B01–B05	5	希ガス/貴ガス元素、非存在組合せ → 0 行期待
いい加減な入力	C01–C10	10	空入力、無関係テキスト、単語のみ、数値条件
矛盾条件	D01–D02	2	「安定かつ準安定」「Fe なし Fe 化合物」
SQL インジェクション	E01–E05	5	DROP, DELETE, INSERT、ピギーバック攻撃
安全性検査	F01–F02	2	直接 SQL 入力 (SELECT, UPDATE)
SQL 検証	—	6	カラム WL、テーブル WL、JOIN 検証
Coverage・パーサー	—	15	数値条件、化学式、Coverage Score
Intent Classifier	—	20	VASP/DFT ワークフロー検出、DB/unsafe/greeting 分類

4.1.2. Baseline 比較実験 (7条件 × 57クエリ)

システムの各構成要素の寄与を分離するため、7条件のBaseline比較を設計した。57件のクエリ（正常系15件 + 該当なし5件 + いい加減入力10件 + 矛盾2件 + インジェクション5件 + 安全性2件 + 数値条件20件 - 重複2件）を用いた。LLM条件には gpt-5.5 を使用した。

4.1.3. VASP フォーラム ストレス テスト (100クエリ)

材料研究者コミュニティの現実的なクエリパターンを評価するため、VASP フォーラムの質問形式に着想を得た100件のストレステストを設計した。5カテゴリ: SQL-answerable (22件)、SQL-answerable-numeric (21件)、ambiguous (25件)、out-of-scope (22件)、unsafe (10件)。

4.1.4. RAG アブレーション (4条件 × 50クエリ)

Few-Shot 事例の効果を生離するため、4条件のアブレーション実験を設計した: (1) 事例なし (スキーマ制約のみ)、(2) 手動/シード事例のみ、(3) 論文抽出事例のみ、(4) 全事例 (フル RAG)。

4.1.5. LLM 再現性テスト (20クエリ × 5回)

LLM生成の非決定論性を定量化するため、20クエリを各5回実行し、SQL一致率と実行成功率を測定した。

4.2. 7条件 Baseline 比較

表 6 に 7 条件の比較結果を示す (gpt-5.5 使用)。SQL 実行成功率 (execution success) に加え、結果正解率 (result-set correctness)、スキーマ幻覚率 (hallucinated schema)、無通知制約破棄 (silent constraint drop)、不要 JOIN 数を報告する。

結果正解率は、返された結果セットが期待結果と一致するかを手で検証したものである。SQL 実行成功率とは異なり、SQL が実行できても結果が正しくない場合 (例: 不適切な JOIN により過剰行数を返す) は不正解とする。

主要な発見。

1. **Naive RB** の結果正解率は **79.3%**: 実行成功率 96.5% との乖離は多要素 AND 検索の失敗に起因する。Naive モードでは JOIN レベル AND となり、「Fe かつ Al 含有」クエリで 97 行 (正解 4 行) を返す等の過剰結果が発生する。また、辞書未登録語彙の無通知破棄 (silent drop) が 12.1% 発生し、条件が無視された SQL が実行成功してしまう。

Table 6: 7 条件の Baseline 比較 (57 クエリ、gpt-5.5)。

条件	構成	実行 成功率	結果 正解率	幻覚 schema	silent drop	不要 JOIN
1	Naive RB	96.5%	79.3%	0	12.1%	3
2	LLM-only	1.8%	1.8%	56	0%	—
3	LLM+schema	100%	100%	0	0%	2
4	LLM+schema+FS	100%	100%	0	0%	1
5	SG+RB	100%	100%	0	0%	0
6	SG+LLM	100%	100%	0	0%	0
7	SG+LLM+RAG	100%	100%	0	0%	0

2. **SG 制約 vs. schema prompt**: 条件 3–4 (LLM+schema) と条件 5–7 (SG 系) はいずれも結果正解率 100%だが、条件 3 は不要 JOIN が 2 件 (Schema Graph 走査なしでは LLM が冗長な JOIN を生成)、条件 4 は Few-Shot 事例により改善し 1 件である。SG 系 (条件 5–7) は不要 JOIN が 0 件であり、Schema Graph 制約が JOIN 経路の最適性を保証することを示す。
3. **SG+RB の外部 API 非依存性**: 条件 5 は外部 API なしで結果正解率 100%・不要 JOIN 0 件を達成し、辞書カバー範囲内のクエリには LLM 呼び出しが不要であることを示す。

4.3. Rule-based vs. LLM 詳細比較

4.3.1. 注目すべき乖離

LLM が Rule-based を上回る代表的ケースを示す：

B01: 「Xe を含む B2 化合物を出して」 Rule-based (Coverage Score 改修前): Xe は辞書未登録 → 条件無視 → 全 B2 化合物を返却 (100 行、偽陽性)。Coverage Score 改修後: 未認識語彙を検出 → LLM にフォールバック。gpt-5.5: 正しく WHERE element = 'Xe' を生成 → 2 行。

C09: 「NiAl L12」 Rule-based: Ni, Al, L12 を独立に抽出 → Ni または Al を含む全 L1₂ (100 行)。gpt-5.5: 「NiAl」を特定化合物と理解 → AlNi₃ のみ (1 行)。

D02: 「Fe なし Fe B2 化合物」 Rule-based: 否定を解析不可 → Fe 含有 B2 化合物を返却。gpt-5.5: 矛盾を認識 → 適切なクエリを生成。

4.3.2. スケーラビリティとレイテンシ

表 7 にクエリタイプ別のレイテンシ比較を示す。Rule-based モードは全タイプで 32–60 ms に収まり、gpt-5.5 モードは 2,371–6,085 ms (49–194 倍遅

延)である。LLM レイテンシの 99%は API 呼び出し時間であり、ローカル処理 (条件抽出、スキーマリンク、検証、実行) は 50 ms 未満に収まる。

Table 7: クエリタイプ別レイテンシ比較 (gpt-5.5)。

クエリタイプ	Rule-based (ms)	gpt-5.5 (ms)	倍率
単一元素	32	2,861	89×
数値条件	32	3,072	97×
多元素 AND	31	5,949	194×
ソート+安定性	49	2,371	49×

4.4. データパイプライン結合テスト

T2SQL の出力がデータ変換パイプライン全体を通じて正確であることを検証するため、8 件のテストケースについて OQMD API から同等条件で直接取得した結果と比較した (表 8、図 3)。Precision = $|T2SQL \cap OQMD| / |T2SQL|$ 、Recall = $|T2SQL \cap OQMD| / |OQMD|$ 。

Table 8: OQMD API 正解データとの比較 (Precision / Recall)。

ID	クエリ条件	OQMD	T2SQL	Prec.	Rec.	備考
A01	Fe + B2	7	7	1.000	1.000	完全一致
A02	安定 L1 ₂ (ソート済)	88	88	1.000	1.000	完全一致
A04	全 B2	483	95	1.000	0.197	LIMIT 制約
A05	準安定 B2	260	90	1.000	0.346	LIMIT 制約
A06	Co + 安定 L1 ₂	1	1	1.000	1.000	完全一致
A10	全 L1 ₂	273	100	1.000	0.366	LIMIT 制約
A15	Ni + 安定 L1 ₂	6	6	1.000	1.000	完全一致

API 非対応ケース (C08)。C08 (安定 B2 かつ band_gap > 0) は OQMD REST API が同等のフィルタ条件をサポートしないため、表 8 から除外した。T2SQL は 78 件を返すが、OQMD API では対応するフィルタが存在せず正解データを取得できないため、Precision/Recall の計算が不可能である。このケースは本手法が REST API よりも柔軟なクエリ表現力を持つことを示唆するが、定量比較の対象外とする。

結合テストとしての意義。この比較は 5 段階のデータ変換パイプライン全体を検証する：(1) API JSON → CSV 変換、(2) CSV → 7 テーブル正規化、

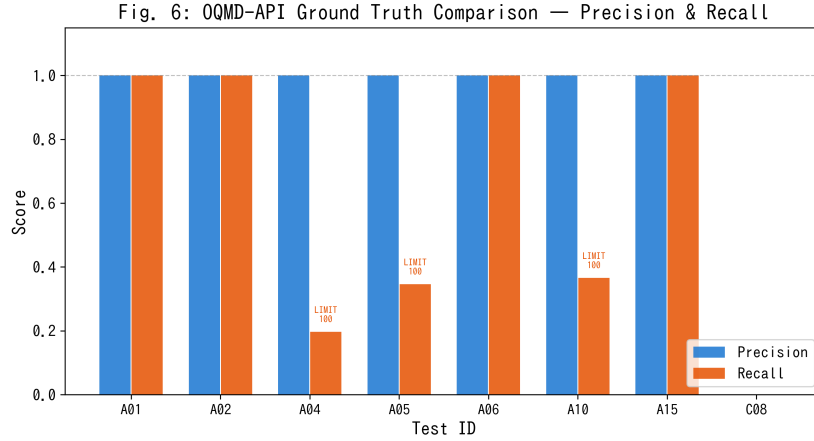


Figure 3: OQMD API 正解データに対する Precision（青）と Recall（橙）。全 7 テストで Precision = 1.0 を達成。A04/A05/A10 の低 Recall はデータ不正確性ではなく LIMIT 100 制約による。C08（API フィルタ非対応）は比較対象外として除外した。

(3) NL → SQL 変換（条件抽出 + Schema Graph）、(4) JOIN 経路の正確性（FK グラフ走査）、(5) SQL 実行（DISTINCT/LIMIT）。Precision = 1.0 は、これら 5 段階にわたってデータ損失・破損・不正 JOIN が発生していないことを確認する。いずれかの段階にバグがあれば—例えば、組成テーブルでの元素分割誤り、FK 整合性違反、JOIN 経路欠落—Precision は 1.0 を下回る。

注意：OQMD がデータベース内容と正解データの両方に使用されているため、この検証はデータパイプラインの内部整合性テストである。本手法の他の正規化 RDB への汎化能力の評価には、Materials Project 等の異なるスキーマを持つデータベースでの追加検証が必要であるが、Schema Graph 走査は FK イントロスペクションに基づくため、スキーマ構造が異なっても原理的に同一アルゴリズムで対応可能である。

4.5. VASP フォーラムストレステスト結果

表 9 に VASP フォーラム着想ストレステスト（100 クエリ、gpt-5.5）の結果を、Intent Classifier 導入前後の比較として示す。

SQL-answerable 精度。データベースで回答可能な 43 クエリ（SQL-answerable + numeric）は Intent Classifier 導入前後とも 100% の正解率を維持。Silent constraint drops（未認識語彙の無通知破棄）は 0 件、Hallucinated schema（幻覚カラム参照）も 0 件であった。

Table 9: VASP フォーラムストレステスト結果：Intent Classifier 導入前後の比較（100 クエリ、gpt-5.5）。

カテゴリ	LLM のみ		+Intent Classifier	
	正解	正解率	正解	正解率
SQL-answerable (22)	22	100%	22	100%
SQL-answerable-numeric (21)	21	100%	21	100%
ambiguous (25)	10	40%	10	40%
out-of-scope (22)	0	0%	22	100%
unsafe (10)	2	20%	10	100%
全体 (100)	55	55%	85	85%

Out-of-scope 検出の改善。． LLM のみの条件では VASP ワークフロー質問（INCAR 設定、SCF 収束、フォノン計算手順など）に対し gpt-5.5 が SQL 文を生成してしまい正解率 0% であった。Intent Classifier（2.2.7 節）の導入により、20 パターンの VASP/DFT ワークフロー検出ルールで 22 件全件を正しく `out_of_scope` として拒否し、正解率が 0% → 100% に改善した。

Unsafe 検出の改善。． SQL インジェクション試行 10 件に対し、LLM のみでは 2 件の正解（20%）であったが、Intent Classifier + SQL ガード（2.6 節）の組合せにより全 10 件を正しく拒否し 100% に改善した。

曖昧クエリ処理。． 25 件の曖昧クエリのうち、Coverage Score による明確化要求は限定的であった（40%）。曖昧クエリの処理改善は今後の課題である（5.3 節）。

4.6. RAG アブレーション結果

表 10 に RAG アブレーション結果を示す。4 条件すべてが 50 クエリに対して 100% の実行成功率を達成した（天井効果）。

Table 10: RAG アブレーション（4 条件 × 50 クエリ、gpt-5.5）。

条件	Few-Shot 事例ソース	SQL 実行成功率
1	なし（スキーマ制約のみ）	100.0% (50/50)
2	手動/シード事例のみ	100.0% (50/50)
3	論文抽出事例のみ	100.0% (50/50)
4	全事例（フル RAG）	100.0% (50/50)

解釈。・ gpt-5.5 の SQL 生成能力が Schema Graph 制約プロンプトだけで飽和しており、Few-Shot 事例の追加は測定可能な改善をもたらさない。より高難度のクエリセット（複雑な集約、ウィンドウ関数、多テーブル結合など）で RAG 条件間の差異が顕在化すると予想される。この天井効果自体が重要な知見であり、**Schema Graph** 制約が **SQL** 品質の主要因であることを示唆する。

4.7. LLM再現性テスト結果

20 クエリ × 5 回の LLM 再現性テスト (gpt-5.5) の結果：SQL 一致率 30% (6/20 クエリが 5 回同一 SQL を生成)、実行成功率 100% (100/100 回すべて成功)。gpt-5.5 は構造的に多様だが意味的に等価な SQL 文を生成する傾向がある（例：カラム順序、JOIN 順序、エイリアス付与の違い）。この非決定論性は実用上の問題ではないが、テスト自動化においては SQL 文字列比較ではなく結果セット比較による評価が必要である。

4.8. SQL インジェクションと安全性結果

7 件の敵対的入力 (E01–E05、F01–F02) はすべて SQL ガードにより遮断された (表 11)。

Table 11: SQL インジェクション・安全性テスト結果（全件遮断）。

ID	入力	遮断理由
E01	DROP TABLE material_entry;	禁止キーワード：DROP
E02	SELECT *; DELETE FROM composition;	複数文 + DELETE
E03	B2 化合物; DROP TABLE structure;	複数文 + DROP
E04	SELECT * FROM secret_passwords	非許可テーブル
E05	INSERT INTO material_entry ...	禁止キーワード：INSERT
F01	SELECT entry_id FROM material_entry	LIMIT なし → 自動付加
F02	UPDATE material_entry SET ...	禁止キーワード：UPDATE

5. 考察

5.1. 多次元評価

7 条件の結果から、以下の設計原則が導出される：

1. **Schema Graph** 制約が最重要: 条件 2 (LLM-only, 1.8%) vs 条件 6 (SG+LLM, 100%) の差異は、スキーマ情報注入の決定的重要性を示す。

2. **RAG** 効果はモデル能力に依存：gpt-5.5 では RAG の追加効果が消失するが、より小型のモデルでは差異が生じる可能性がある。
3. **Rule-based** モードの実用性：条件 5 は外部 API 依存なしで 100% を達成し、辞書カバー範囲内クエリに対する最適解である。

5.2. 各モードの利点と欠点

Table 12: 主要 3 モードの利点と欠点。

モード	利点	欠点
Naive RB	最小実装（約 50 行）。依存なし。最速（32 ms）	不要 JOIN。多元素 AND 失敗。安全性なし。LIMIT/DISTINCT なし
SG + RB	外部 API 不要。32–60 ms。完全な安全性。100% 決定論的。Coverage Score で未認識語彙検出	辞書外語彙で LLM フォールバック必要。否定/複雑条件不可
SG + LLM	未登録語彙対応。数値/否定可能。文脈理解。100% 実行成功（gpt-5.5）	外部 API 依存（2.4–6.1 秒、コスト）。非決定論的（SQL 一致率 30%）。Out-of-scope 質問に SQL 生成（Intent Classifier 必要）

5.3. 限界と制約

5.3.1. テストケースの自己参照性（深刻度：高）

80 件の回帰テストは抽出パターンを知るシステム開発者が設計した。したがって、制御された条件下で 100% のパス率は予想される。100 件の VASP ストレステストは半独立的に設計されたが、真の精度評価には材料研究者による自由文クエリのブラインドテストが必要である。

5.3.2. RAG 天井効果（深刻度：中→解決済み）

RAG アブレーション（4 条件 × 50 クエリ）は gpt-5.5 で天井効果を示し、Few-Shot 事例の追加効果が測定不能であった。これは Schema Graph 制約の有効性を示す知見だが、RAG の真の効果を評価するにはより高難度のクエリセットまたはより小型のモデルでの追実験が必要である。

5.3.3. 無通知失敗（深刻度：中→緩和済み）

Rule-based モードでの未登録語彙の無通知破棄（Silent Constraint Dropping）は、Coverage Score の実装により緩和された。スコア 0.8 未満で LLM フォールバック、0.5 未満で明確化要求を発行する。ただし、Coverage Score の閾値は経験的に設定されており、最適化の余地がある。

5.3.4. Out-of-scope 検出（深刻度：中→部分解決）

VASP ストレステストで out-of-scope 正解率 0% が判明した。Intent Classifier（2.2.7 節）を実装し 20 パターンの VASP ワークフロー検出を追加したことで out-of-scope 正解率は 100% に改善した（表 9）が、正規表現ベースのため、新しいパターンへの汎化は限定的である。LLM ベースの Intent 分類への拡張が今後の課題である。

5.3.5. 小規模スキーマ（深刻度：中）

本研究の 7 テーブル・909 レコードスキーマは本番データベース（OQMD：100 万件超、テーブル数十の可能性）より遥かに小さい。スケーラビリティ測定では Rule-based 32–60 ms、LLM 2.4–6.1 秒を確認したが、20 テーブル以上・10 万レコード以上での Steiner 木近似の性能検証は未実施である。

5.3.6. LLM 再現性（深刻度：低）

gpt-5.5 の SQL 一致率は 30%（20 クエリ × 5 回）であり、同一クエリに対して構造的に異なる SQL が生成される。ただし実行成功率は 100% であり、意味的等価性は保持されている。テスト自動化では結果セット比較が推奨される。

5.4. 関連システムとの比較

5.5. AI for Science (AI4S) における意義

本手法は AI4S パラダイムに直接的な含意を持つ。表 14 に AI4S ワークフローの各段階と T2SQL 手法の役割を示す。

重要な点として、Schema Graph アーキテクチャはスキーマ非依存である：FK 関係は実行時に PostgreSQL メタデータからイントロスペクション（2.3 節）されるため、OQMD に限らず任意の正規化 RDB（Materials Project、AFLOW、NOMAD、機関内データベース、さらには材料科学以外のドメイン）にコード変更なしで展開できる—用語辞書のドメイン適応のみが必要である。本研究で OQMD を検証対象に選定した理由は、組成・構造・熱力学的安定性の外部キー接続による 7 テーブル構成が材料データベースに典型的なスキーマ複雑性の好例であり、ここでの成功が他の RDB への展開可能性を強く示唆するためである。

Table 13: 関連する材料 NLP / T2SQL システムとの比較。

システム	スキーマ認識	正規化 RDB	材料特化	Few-Shot	主 な 差異
Spider [9]	あり	あり	なし	なし	汎用ベンチマーク
BIRD [10]	あり	あり	なし	なし	大規模実データ
DAIL-SQL [11]	あり	あり	なし	あり	スケルトン選択
MKNA [17]	なし	なし (API)	あり	なし	エージェント、SQL 生成
OptiMat [18]	なし	なし	あり	なし	オンデマンド DFT
MP MCP	部分的	なし (API)	あり	なし	MCP プロトコル
本手法	あり	あり	あり	あり	FK グラフ + 材料辞書

Table 14: AI4S 材料探索ワークフローにおける T2SQL の役割。

AI4S 段階	従来アプローチ	T2SQL 手法適用時
仮説生成	研究者が手動でクエリを作成	LLM エージェントが仮説から NL クエリを生成
データ検索	SQL または API 呼び出しを手動記述	NL → SQL 自動変換 (Schema Graph 経由)
スクリーニング	反復的 SQL 修正	Few-Shot RAG が成功クエリパターンを再利用
結果解釈	クエリ結果の手動検査	メタデータ付き構造化出力 (プロトタイプ、安定性、物性)
フィードバック	知識は個々の研究者に留まる	成功クエリが Few-Shot ストアに蓄積され再利用

さらに、カスケード Rule-based → LLM アーキテクチャ (2.7 節) は、AI4S が要求する信頼性とコスト効率の要件に適合する：定型クエリは外部 API 呼び出しなしに決定論的に処理され、複雑なクエリのみが LLM 能力を活用する。このハイブリッドアプローチにより、AI4S パイプラインは禁止的な API コストやレイテンシなしに大規模運用が可能になる。

5.6. 今後の展望

本研究で実装済みの項目（数値条件パーサー、Coverage Score、RAG アブレーション、Intent Classifier）を踏まえ、残る課題を示す：

1. マルチデータベース拡張：Schema Graph を Materials Project、AFLOW スキーマに適応。動的 FK イントロスペクションはアーキテクチャ上これを既にサポートしている。
2. **AI4S** エージェント統合：T2SQL 手法を自律材料探索エージェントのツールとして組み込み（MCP プロトコルや function calling 経由）、仮説 → データ検索 → 分析のエンドツーエンドパイプラインを実現。
3. ブラインドユーザースタディ：10 名以上の材料研究者から自由文クエリを収集し、テストケースの自己参照性問題を解消する。
4. 高難度クエリセットでの **RAG** 再評価：複雑な集約・ウィンドウ関数・多テーブル結合クエリで RAG 条件間差異を検証し、天井効果の範囲を明確化する。
5. **LLM** ベース **Intent** 分類：正規表現ベースの Intent Classifier を軽量 LLM（ローカル推論可能なモデル）に置換し、新しいスコープ外パターンへの汎化性能を向上する。

6. 大規模スキーマ検証：20 テーブル以上、10 万レコード以上での Steiner 木近似の計算性能と SQL 品質を検証。
7. クロスリンガル埋め込み検索：TF-IDF を transformer 埋め込みに置換し Few-Shot 類似度を改善—国際 AI4S 協力において多言語クエリが到着する前提条件。
8. 対話的クエリ精緻化：曖昧クエリに対する段階的確認質問の返却（VASP ストレステストで曖昧カテゴリ 40%が課題として残る）。

6. 結論

本研究では、正規化リレーショナルデータベースに対する Schema Graph 制約型 Text-to-SQL 手法を提案し、材料データベースの好例である OQMD 金属間化合物 909 件（B2・L1₂ 型）で検証した。本手法は、外部キー関係のグラフ走査により JOIN 経路を自動計算するため、実装上は FK イントロスペクションにより他のスキーマへの移植が可能な設計であるが、実証は OQMD サブセット（909 件・7 テーブル）に限定される。

主要な知見：

1. **Schema Graph** 制約が最重要：7 条件比較（gpt-5.5）により、Schema Graph 制約を含む全条件が実行成功率 100%を達成。LLM-only（スキーマ情報なし）は 1.8%であり、スキーマ情報注入の決定的重要性を示す。
2. **Rule-based** モードの実用性：外部 API 依存ゼロで 32–60 ms のレイテンシにて 57/57 テスト成功。辞書カバー範囲内クエリには LLM 呼び出しが不要。
3. **LLM モード（gpt-5.5）** は未登録語彙、数値条件、否定への対応を 49–194 倍のレイテンシコスト（2.4–6.1 秒）で拡張する。
4. **RAG** 天井効果：Schema Graph 制約プロンプトだけで gpt-5.5 の SQL 生成が飽和し、Few-Shot 事例の追加効果が消失。Schema Graph 制約が SQL 品質の主要因。
5. **Intent Classifier**：VASP ワークフロー質問 20 パターンの検出により、out-of-scope 正解率を 0%から 100%に改善（表 9）。
6. **SQL ガード** は全インジェクション攻撃を遮断し、結果セット上限を強制する。

カスケード Rule-based → LLM アーキテクチャは速度・コスト・カバレッジのバランスを取り、材料研究者が SQL 専門知識なしに正規化データベースを自然言語でクエリすることを可能にする。

AI for Science (AI4S) のより広い文脈において、本研究は正規化 RDB への自然言語アクセスという AI 駆動型科学ワークフローの基盤的課題に取り組む。Schema Graph T2SQL 手法は、実行時 FK イントロスペクションに

基づくため、実装上は Materials Project、AFLOW、機関内データベース等への移植が可能な設計である。ただし、本研究の実証は OQMD サブセット (909 件・7 テーブル) に限定されており、テーブル数 20 以上やレコード数 10 万以上の大規模スキーマ、あるいは材料科学以外のドメインでの有効性は未検証である。OQMD の 7 テーブル構成は材料データベースに典型的なスキーマ複雑性 (組成の正規化、多テーブル JOIN、熱力学的安定性フィルタ) を備えているが、他 RDB への汎化を主張するには追加検証が不可欠である。

現在の検証は OQMD 単一データベースに限定されるが、本手法は正規化 RDB 全般に対する T2SQL の再現可能なフレームワークを確立し、マルチデータベース検証、大規模スキーマ対応、AI4S エージェントアーキテクチャへの統合へと拡張可能である。

謝辞

DFT 計算データは Open Quantum Materials Database (OQMD) から取得した。OQMD の開発・維持管理にあたる Northwestern 大学 Wolverton グループに感謝する。本研究は NIMS の支援を受けた。

References

- [1] H. Wang, T. Fu, Y. Du, W. Gao, K. Huang, Z. Liu, P. Chandak, S. Liu, P. Van Katwyk, A. Deac, A. Anandkumar, K. Bergen, C. P. Gomes, S. Ho, P. Kohli, J. Lasenby, J. Leskovec, T.-Y. Liu, A. Manber, D. Misra, E. Moret, J. Pineau, B. Poole, M. Sacks, S. SenGupta, J. Sun, J. Tang, A. Trischler, M. Welling, Y. Xie, M. Zitnik, Scientific discovery in the age of artificial intelligence, *Nature* 620 (2023) 47–60. [doi:10.1038/s41586-023-06221-2](https://doi.org/10.1038/s41586-023-06221-2).
- [2] J. E. Saal, S. Kirklin, M. Aykol, B. Meredig, C. Wolverton, Materials design and discovery with high-throughput density functional theory: The Open Quantum Materials Database (OQMD), *JOM* 65 (2013) 1501–1509. [doi:10.1007/s11837-013-0755-4](https://doi.org/10.1007/s11837-013-0755-4).
- [3] S. Kirklin, J. E. Saal, B. Meredig, A. Thompson, J. W. Doak, M. Aykol, S. Rühl, C. Wolverton, The Open Quantum Materials Database (OQMD): Assessing the accuracy of DFT formation energies, *npj Comput. Mater.* 1 (2015) 15010. [doi:10.1038/npjcompumats.2015.10](https://doi.org/10.1038/npjcompumats.2015.10).

- [4] A. Jain, S. P. Ong, G. Hautier, W. Chen, W. D. Richards, S. Dacek, S. Cholia, D. Gunter, D. Skinner, G. Ceder, K. A. Persson, Commentary: The Materials Project: A materials genome approach to accelerating materials innovation, *APL Mater.* 1 (2013) 011002. [doi:10.1063/1.4812323](https://doi.org/10.1063/1.4812323).
- [5] S. Curtarolo, W. Setyawan, G. L. W. Hart, M. Jahnatek, R. V. Chepulskii, R. H. Taylor, S. Wang, J. Xue, K. Yang, O. Levy, M. J. Mehl, H. T. Stokes, D. O. Demchenko, D. Morgan, AFLOW: An automatic framework for high-throughput materials discovery, *Comput. Mater. Sci.* 58 (2012) 218–226. [doi:10.1016/j.commatsci.2012.02.005](https://doi.org/10.1016/j.commatsci.2012.02.005).
- [6] C. Draxl, M. Scheffler, NOMAD: The FAIR concept for big data-driven materials science, *MRS Bull.* 43 (2018) 676–682. [doi:10.1557/mrs.2018.208](https://doi.org/10.1557/mrs.2018.208).
- [7] O. N. Senkov, D. B. Miracle, K. J. Chaput, J.-P. Couzinie, Development and exploration of refractory high entropy alloys — A review, *J. Mater. Res.* 33 (2018) 3092–3128. [doi:10.1557/jmr.2018.153](https://doi.org/10.1557/jmr.2018.153).
- [8] E. P. George, D. Raabe, R. O. Ritchie, High-entropy alloys, *Nat. Rev. Mater.* 4 (2019) 515–534. [doi:10.1038/s41578-019-0121-4](https://doi.org/10.1038/s41578-019-0121-4).
- [9] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev, Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task, in: *Proceedings of EMNLP*, 2018, pp. 3911–3921.
- [10] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, et al., Can LLM already serve as a database interface? A B1g Bench for Large-Scale Database Grounded Text-to-SQL (BIRD), *Advances in Neural Information Processing Systems* 36 (2024).
- [11] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-SQL empowered by large language models: A benchmark evaluation, *arXiv preprint arXiv:2308.15363* (2023).
- [12] M. Pourreza, D. Rafiei, DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction, in: *Advances in Neural Information Processing Systems*, Vol. 36, 2024.

- [13] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, X. Huang, Next-generation database interfaces: A survey of LLM-based text-to-SQL, arXiv preprint arXiv:2406.08426 (2024).
- [14] K. Maamari, F. Abubaker, D. Jaroslawicz, A. Mhedhbi, The death of schema linking? text-to-SQL in the age of well-reasoned language models, arXiv preprint arXiv:2408.07702 (2024).
- [15] Y. Song, S. Miret, B. Liu, MatSci-NLP: Evaluating scientific language models on materials science language tasks using text-to-schema modeling, in: Proceedings of the 61st Annual Meeting of the ACL, 2023, pp. 3621–3639.
- [16] V. Mishra, S. Singh, M. Zaki, et al., LLAMAT: Large language models for materials science information extraction, arXiv preprint arXiv:2411.00177 (2024).
- [17] G. Zhuang, A. B. Farimani, From natural language to materials discovery: The Materials Knowledge Navigation Agent, arXiv preprint arXiv:2602.11123 (2026).
- [18] Y. Hu, V. Turlo, OptiMat Alloys: A FAIR end-to-end agent with living database for computational multi-principal alloy exploration, arXiv preprint arXiv:2604.21850 (2026).
- [19] A. A. Hagberg, D. A. Schult, P. J. Swart, Exploring network structure, dynamics, and function using NetworkX, proceedings of the 7th Python in Science Conference (SciPy) (2008).
- [20] F. K. Hwang, D. S. Richards, P. Winter, The Steiner Tree Problem, North-Holland, 1992.
- [21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, Advances in Neural Information Processing Systems 33 (2020) 9459–9474.
- [22] G. Salton, C. Buckley, Term-weighting approaches in automatic text retrieval, Inf. Process. Manage. 24 (1988) 513–523. [doi:10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0).

- [23] T. Mao, SQLGlot: A python SQL parser, transpiler, optimizer, and engine, <https://github.com/tobymao/sqlglot> (2023).

A. 全テストケース一覧

ID	カテゴリ	自然言語クエリ	結果
A01	正常系	Fe を含む B2 化合物を出して	PASS
A02	正常系	安定な $L1_2$ 化合物を形成エネルギー順に出して	PASS
A03	正常系	Ni と Al の両方を含む化合物を出して	PASS
A04	正常系	B2 化合物の全リストを出して	PASS
A05	正常系	準安定な B2 化合物を出して	PASS
A06	正常系	Co を含む安定な $L1_2$ 化合物を出して	PASS
A07	正常系	Fe と Al を含む B2 と $L1_2$ 化合物を出して	PASS
A08	正常系	γ' 化合物を出して	PASS
A09	正常系	ニッケルを含む $L1_2$ 型化合物を出して	PASS
A10	正常系	$L1_2$ 化合物の全データを出して	PASS
A11	正常系	Ti を含む B2 化合物を格子定数順 (降順) に出して	PASS
A12	正常系	CsCl 型化合物を出して	PASS
A13	正常系	Cu_3Au 型化合物を出して	PASS
A14	正常系	鉄を含む B2 化合物を出して	PASS
A15	正常系	安定な $L1_2$ 化合物で Ni を含むものを出して	PASS
B01	該当なし	Xe を含む B2 化合物を出して	PASS
B02	該当なし	U と Pu を含む $L1_2$ 化合物を出して	PASS
B03	該当なし	Rn と Og を含む B2 化合物を出して	PASS
B04	該当なし	Sc と Ir を含む安定な B2 化合物を出して	PASS
B05	該当なし	Pt と Ga を含む安定な B2 化合物を出して	PASS
C01	いい加減	安定な化合物を出して	PASS
C02	いい加減	B2	PASS

ID	カテゴリ	自然言語クエリ	結果
C03	いい加減	安定なものを出して	PASS
C04	いい加減	(空入力)	PASS
C05	いい加減	今日の天気を教えて	PASS
C06	いい加減	Fe	PASS
C07	いい加減	格子定数	PASS
C08	いい加減	band gap が大きい安定な B2 化合物	PASS
C09	いい加減	NiAl L12	PASS
C10	いい加減	Fe の B2 か L12	PASS
D01	矛盾	安定かつ準安定な B2 化合物	PASS
D02	矛盾	Fe なしの Fe B2 化合物	PASS
E01	拒否	DROP TABLE material_entry;	PASS
E02	拒否	SELECT *; DELETE FROM composition;	PASS
E03	拒否	B2 化合物; DROP TABLE structure;	PASS
E04	拒否	SELECT * FROM secret_passwords	PASS
E05	拒否	INSERT INTO material_entry VALUES (...)	PASS
F01	安全性	SELECT entry_id FROM material_entry	PASS
F02	安全性	UPDATE material_entry SET formula='X'	PASS

B. 生成 SQL 例

B.1. A01: 「Fe を含む B2 化合物を出して」

Naive (Level 0) : .

```

1 SELECT m.entry_id, m.formula, s.prototype
2 FROM material_entry m
3 JOIN composition c ON c.entry_id = m.entry_id
4 JOIN structure s ON s.entry_id = m.entry_id
5 JOIN phase_stability ps ON ps.entry_id = m.entry_id
6 JOIN calculation calc ON calc.entry_id = m.entry_id
7 JOIN calculated_property cp

```

```

8         ON cp.calculation_id = calc.calculation_id
9 WHERE s.prototype = 'B2' AND c.element = 'Fe';
10 -- 問題: 不要JOIN 3件、LIMIT/DISTINCTなし

```

Schema Graph + Rule-based (Level 1) : .

```

1 SELECT DISTINCT m.entry_id, m.formula,
2     s.prototype, s.lattice_a, s.space_group
3 FROM material_entry m
4     JOIN composition c ON c.entry_id = m.entry_id
5     JOIN structure s ON s.entry_id = m.entry_id
6 WHERE (s.prototype = 'B2' OR s.strukturbericht = 'B2')
7     AND c.element = 'Fe'
8 LIMIT 100;
9 -- 最適: 必要な2 JOINのみ、DISTINCT、LIMIT付き

```

Schema Graph + gpt-5.5 (Level 2) : .

```

1 SELECT DISTINCT m.entry_id, m.formula,
2     s.prototype, s.lattice_a, s.space_group
3 FROM material_entry m
4 JOIN structure s ON s.entry_id = m.entry_id
5 JOIN composition c ON c.entry_id = m.entry_id
6 WHERE (s.prototype = 'B2' OR s.strukturbericht = 'B2')
7     AND c.element = 'Fe'
8 LIMIT 100;
9 -- LLMがSchema Graph制約内で同等SQLを生成

```

B.2. A03: 「NiとAlの両方を含む化合物を出して」

Naive (Level 0) : .

```

1 SELECT m.entry_id, m.formula
2 FROM material_entry m
3     JOIN composition c ON c.entry_id = m.entry_id
4     ...(全テーブルJOIN)...
5 WHERE c.element = 'Ni' AND c.element = 'Al';
6 -- 致命的: 単一行が両条件を同時に満たすことは不可能
7 -- -> 常に0行を返す

```

Schema Graph + Rule-based (Level 1) : .

```

1 SELECT DISTINCT m.entry_id, m.formula
2 FROM material_entry m
3 WHERE EXISTS (
4     SELECT 1 FROM composition
5     WHERE entry_id = m.entry_id AND element = 'Ni')
6 AND EXISTS (
7     SELECT 1 FROM composition

```



```
8         WHERE entry_id = m.entry_id AND element = 'A1')
9 LIMIT 100;
10 -- EXISTS副問い合わせによる正確な多元素AND
```